



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Сервер для отдачи статического содержимого»

Студент ИУ7-74Б

(Подпись, дата) **И. С. Бугаков**
(И.О.Фамилия)

Руководитель

(Подпись, дата) **М. Д. Мицевич**
(И.О.Фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Протокол HTTP	5
1.1.1 Структура HTTP-запроса	5
1.1.2 Методы HTTP-запросов	5
1.1.3 Структура HTTP-ответа	6
1.1.4 Коды состояния HTTP	6
1.2 Мультиплексирование ввода-вывода	6
1.3 Использование пула процессов-обработчиков	7
2 Конструкторская часть	8
2.1 Алгоритм работы процесса-предка	8
3 Технологическая часть	14
3.1 Выбор средств реализации	14
3.2 Реализация алгоритмов	14
3.3 Демонстрация работы алгоритма	27
3.4 Тестирование	28
4 Исследовательская часть	31
4.1 Характеристики устройства	31
4.2 Методика исследования	31
4.3 Результаты	31
4.4 Вывод	34
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37

ВВЕДЕНИЕ

HTTP-сервер для отдачи статических ресурсов является основным компонентом сетевой инфраструктуры. Он отвечает за выдачу неизменяемых ресурсов (HTML, CSS, JavaScript, изображения, файлы). Производительность такого сервера напрямую влияет на время отклика и суммарную нагрузку на вычислительные ресурсы.

Для обслуживания большого количества активных соединений серверу необходимо эффективно работать с сетевыми соединениями. Один из классических подходов — неблокирующий ввод-вывод и мультиплексирование по готовности дескрипторов — позволяет обслуживать множество соединений в рамках одного процесса.

Цель работы — разработать HTTP-сервер для отдачи статического содержимого с использованием мультиплексирования соединений на основе системного вызова `select()` и предварительно созданных процессов-обработчиков (worker'ов), обеспечивающих параллельную обработку соединений.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области: основные принципы устройства и работы протокола HTTP, механизм мультиплексирования с использованием системного вызова `select()` и подход к организации параллельной обработки соединений;
- разработаны алгоритмы обработки HTTP-запросов и обслуживания множества соединений одним процессом-обработчиком;
- выбрать средства реализации и реализовать HTTP-сервер;
- провести исследование и определить: максимальное количество обслуживаемых сетевых соединений, зависимость скорости передачи данных по каждому сетевому соединению, совокупной средней скорости передачи и среднего времени обработки запроса.

1 Аналитическая часть

В данной части приводятся основы устройства и работы протокола HTTP и подход к обслуживанию множества клиентов сервером отдачи статического содержимого с использованием мультиплексирования ввода-вывода посредством системного вызова `select()` и предварительно созданного набора процессов-обработчиков (`worker`'ов).

1.1 Протокол HTTP

HTTP (HyperText Transfer Protocol) — протокол клиент-серверного взаимодействия прикладного уровня для получения ресурсов, используемый в распределенных гипертекстовых системах [3]. В данном протоколе соединение и запросы к серверу инициируются получателем, сервер формирует ответ и отправляет его получателю [9].

1.1.1 Структура HTTP-запроса

HTTP-запрос состоит из трех основных частей:

- 1) стартовая строка (`request-line`) — метод, целевой ресурс, версия протокола разделенные одиночными пробелами;
- 2) заголовки (`headers`) — ноль или более строк в формате `field-name : field-value`;
- 3) тело сообщения (`body`) — необязательная часть, которая обычно отсутствует в GET или HEAD запросах.

1.1.2 Методы HTTP-запросов

Всего определено 8 методов HTTP-запросов [2]:

- GET — получение содержимое целевого ресурса;
- HEAD — то же что и GET, но без передачи сервером тела ответа;
- POST — выполнение специфичной для данного целевого ресурса обработки содержимого запроса;
- PUT — замена всех текущих представлений целевого ресурса содержимым запроса;
- DELETE — удаление всех текущих представлений целевого ресурса;
- CONNECT — установление соединения с сервером, определяемым целевым ресурсом;
- OPTIONS — получение разрешенных методов и заголовков для взаимодействия с целевым ресурсом;
- TRACE — получение запроса в том виде, в котором он был доставлен серверу.

Обязательными для реализации серверами общего назначения являются только методы

GET и HEAD [2].

1.1.3 Структура HTTP-ответа

HTTP-ответ также состоит из трех основных частей, аналогичных частям HTTP-запроса:

- 1) строка состояния (status-line) — версия протокола, код состояния, необязательное текстовое описание кода состояния разделенные одиночными пробелами;
- 2) заголовки (headers) — ноль или более строк в том же формате, что и заголовки HTTP-запроса;
- 3) тело сообщения (body) — необязательная часть, которая отсутствует в HEAD запросах и может также отсутствовать в GET запросах.

1.1.4 Коды состояния HTTP

Выделяют 5 классов кодов состояния HTTP [2]:

- 1xx — информационный, указывает на промежуточный ответ, для сообщения о состоянии соединения или ходе выполнения запроса до отправки окончательного ответа;
- 2xx — успешный, указывает на то, что запрос был успешно получен, понят и обработан;
- 3xx — перенаправление, указывает на то, какие дальнейшие действия необходимо предпринять клиенту для выполнения запроса;
- 4xx — ошибка клиента, указывает на то, что запрос клиента выглядит ошибочным;
- 5xx — ошибка сервера, указывает на то, что сервер допустил ошибку в ходе выполнения данного запроса, или не способен обработать данный метод.

В описываемом в данной работе сервере используются коды состояния: 200 OK, 403 Forbidden, 404 Not Found, 405 Method Not Allowed.

1.2 Мультиплексирование ввода-вывода

Системный вызов `select()` позволяет программе отслеживать несколько файловых дескрипторов, ожидая, пока один или несколько из них не станут готовыми для какой-либо операции ввода-вывода. Файловый дескриптор считается готовым, если возможно выполнить соответствующую операцию ввода-вывода без блокировки. Таким образом системный вызов `select()` позволяет осуществлять мультиплексирование ввода-вывода [8].

Основным недостатком системного вызова `select()` является то, что он позволяет отслеживать только файловые дескрипторы, номера которых меньше `FD_SETSIZE` (1024).

1.3 Использование пула процессов-обработчиков

Архитектура с использованием многопроцессной обработки запросов предполагает, что родительский процесс создает набор дочерних процессов, посредством системного вызова `fork()` [7], до получения каких-либо запросов, после чего процессы-потомки обрабатывают получаемые запросы, а процесс-предок отслеживает их состояние и создает новый дочерний процесс, в случае, если один из дочерних процессов завершился.

Преимуществом данного подхода является изолированность процессов-обработчиков друг от друга. К недостаткам можно отнести: высокое потребление памяти, так как каждый обработчик представляет собой отдельный процесс; сложность балансировки нагрузки; проблема *thundering-herd*, когда несколько обработчиков просыпаются для выполнения системного вызова `accept()` [6].

Вывод

В данном разделе были рассмотрены основы устройства и работы протокола HTTP и подход к обработке множества клиентов с помощью мультиплексирования ввода-вывода системным вызовом `select()` и предварительно созданного набора процессов-обработчиков.

2 Конструкторская часть

В данном разделе представлены схемы алгоритмов разработанного сервера для отдачи статического содержимого. Показаны алгоритмы работы процесса-предка, работы процесса-обработчика, обработки стартовой строки HTTP-запроса и формирования плана HTTP-ответа, включающего строку состояния и заголовки.

2.1 Алгоритм работы процесса-предка

На рисунке 2.1 представлена схема алгоритма работы процесса-предка.

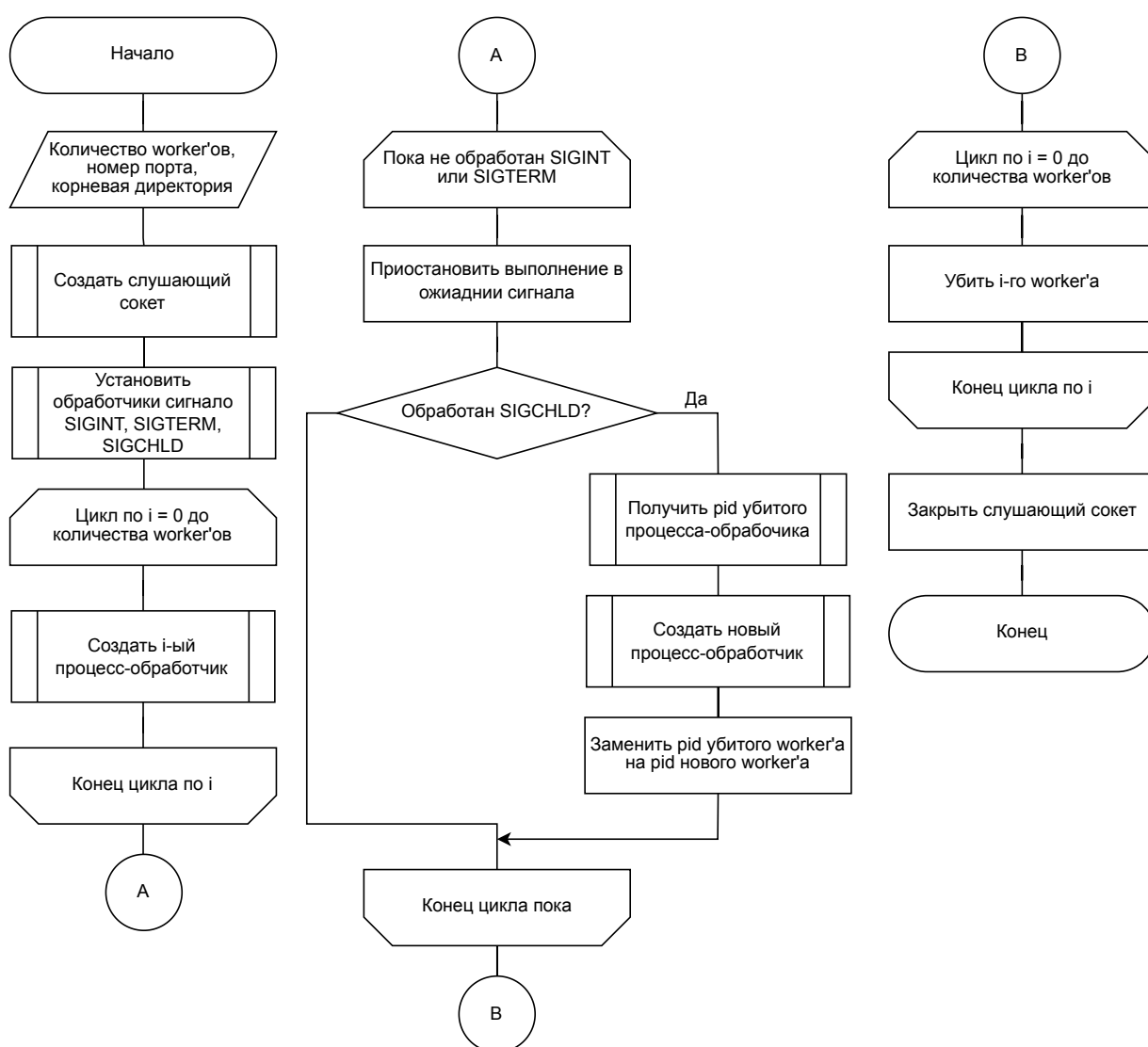


Рисунок 2.1 — Схема алгоритма работы процесса-предка

На рисунке 2.2 приведена схема алгоритма работы процесса-обработчика. Процесс-обработчик ожидает запросов на соединение, принимает их, осуществляет отправку статического ресурса по частям, после полного прочтения запроса.

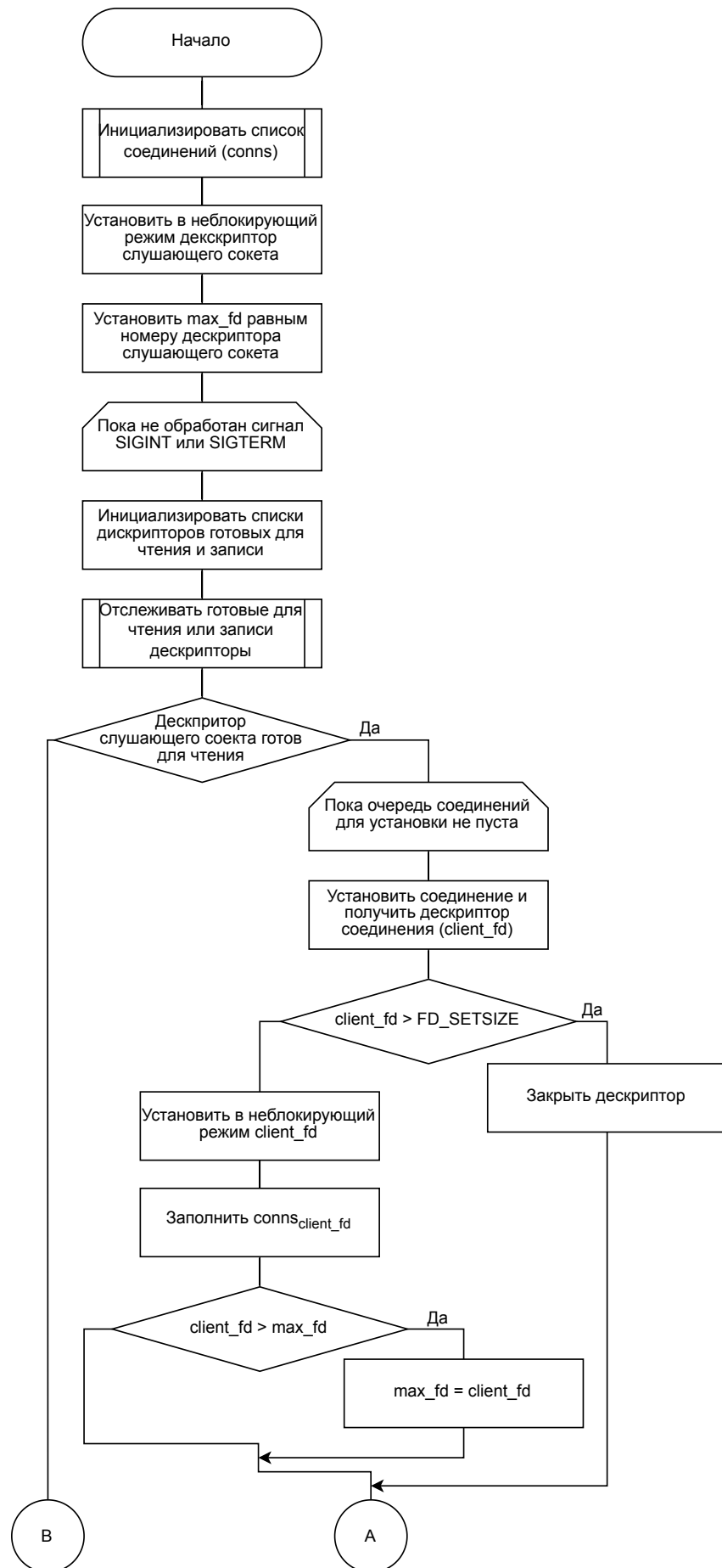


Рисунок 2.2 — Схема алгоритма работы процесса-обработчика

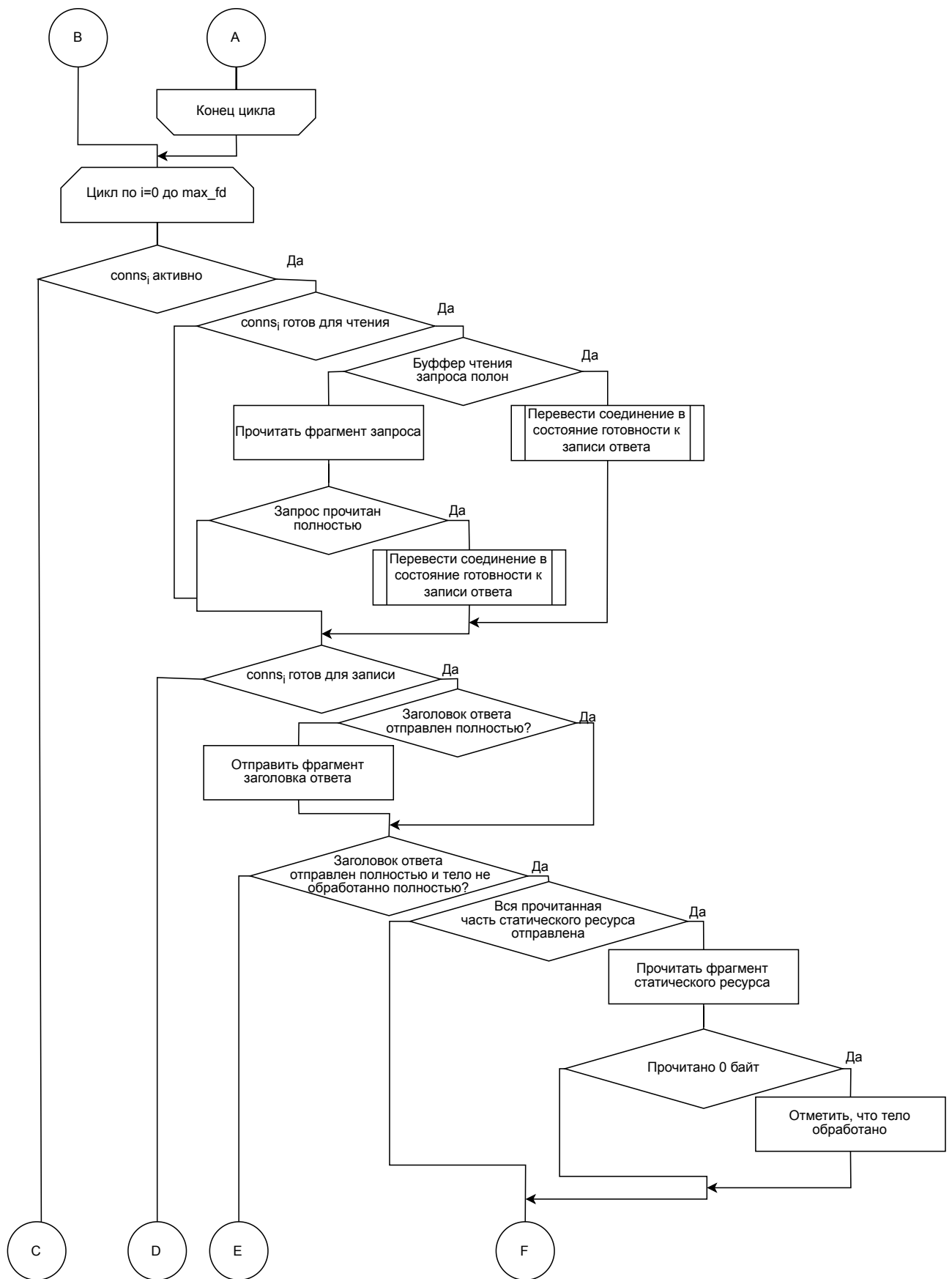


Рисунок 2.2 — Схема алгоритма работы процесса-обработчика

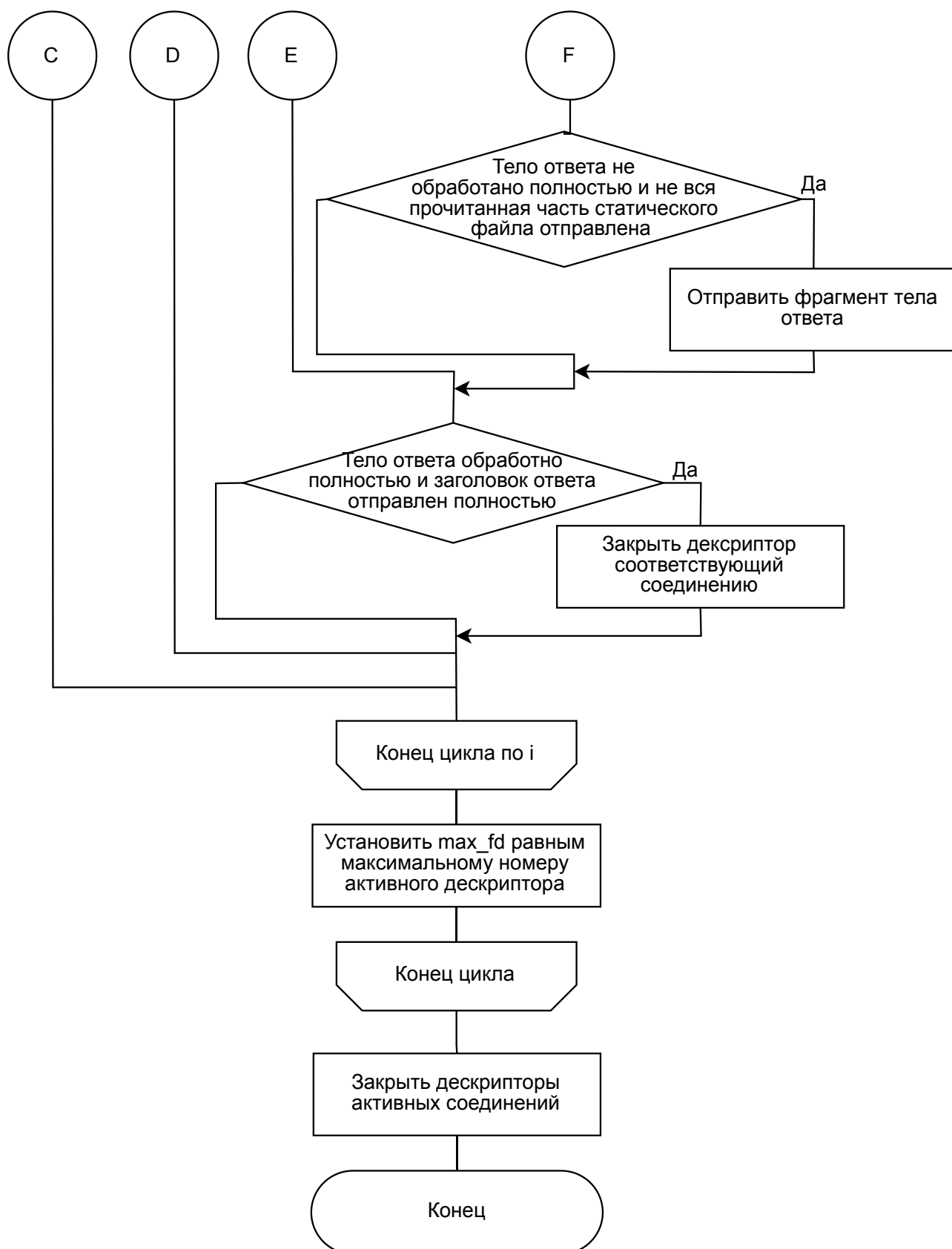


Рисунок 2.2 — Схема алгоритма работы процесса-обработчика

На рисунке 2.2 представлена схема алгоритма разбора стартовой строки HTTP-запроса.

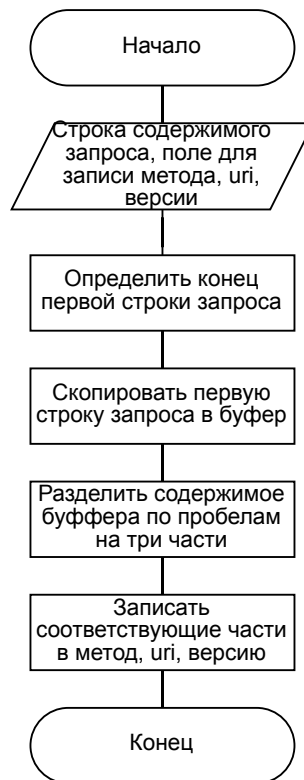


Рисунок 2.2 — Схема алгоритма разбора стартовой строки HTTP-запроса

На рисунке 2.2 представлена схема алгоритма формирования плана HTTP-ответа.

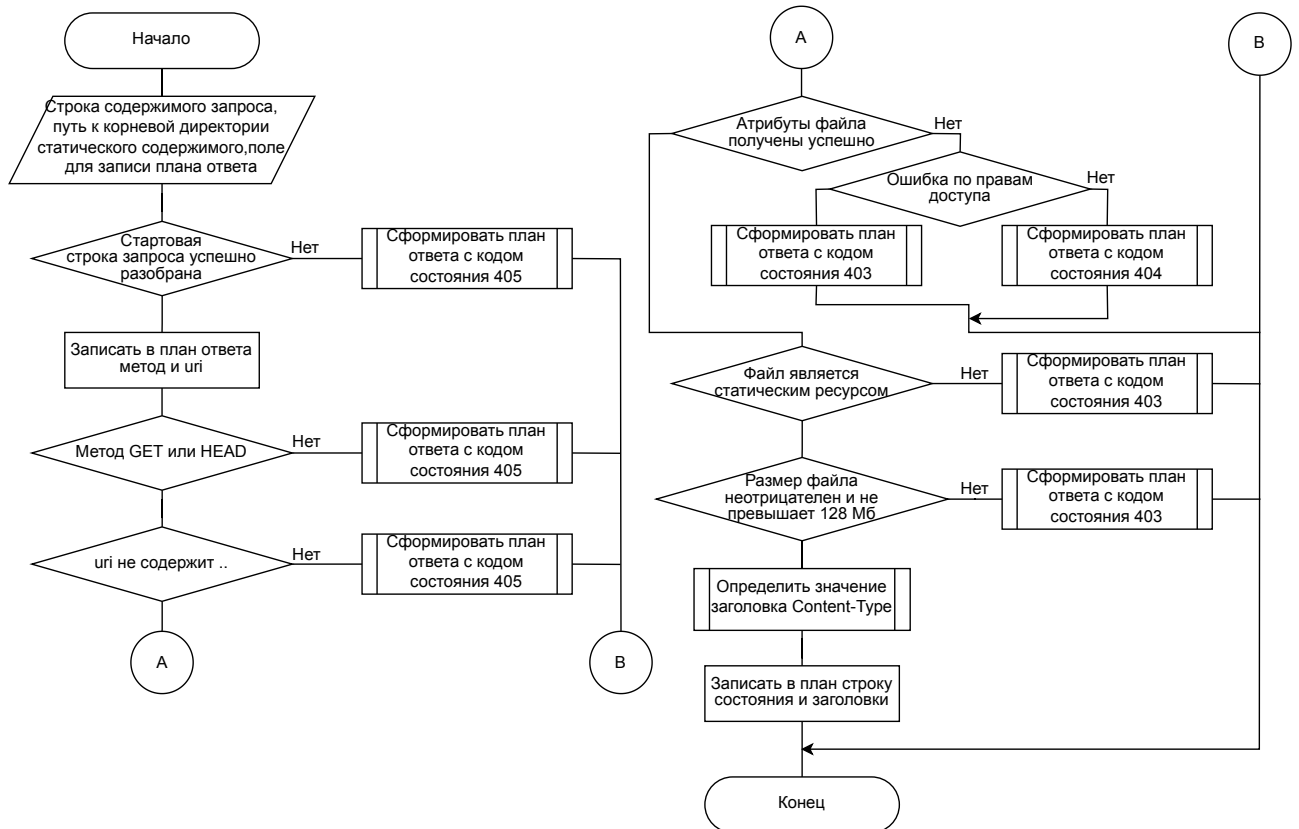


Рисунок 2.2 — Схема алгоритма формирования плана HTTP-ответа

Вывод

В данном разделе спроектированы процедуры работы основного процесса-предка и процессов-обработчиков, а также алгоритмы разбора стартовой строки HTTP-запроса и формирования плана HTTP ответа, приведены соответствующие схемы алгоритмов.

3 Технологическая часть

В данном разделе представлены выбранные средства реализации, примеры запуска реализованного HTTP-сервера и выполнения функциональных тестов.

3.1 Выбор средств реализации

В качестве целевой операционной системы была выбрана Ubuntu 22.04 LTS [1], т. к. данная операционная система предоставляет поддержку необходимых функций ядра, описанных стандартом POSIX, а также в связи с наличием утилиты `wrk` [10], используемой для нагрузочного тестирования HTTP-сервера.

Для реализации HTTP-сервера был выбран язык программирования C [5], т. к. данный язык обеспечивает доступ к необходимым системным вызовам посредством библиотеки `glibc` [4].

3.2 Реализация алгоритмов

Ниже приведены листинги основных алгоритмов HTTP-сервера:

- листинг 3.1 — алгоритм работы процесса-предка;
- листинг 3.2 — обработчики сигналов и их установка;
- листинг 3.3 — создание процессов-обработчиков;
- листинг 3.4 — создание слушающего сокета;
- листинг 3.5 — алгоритм работы `worker`'а;
- листинг 3.6 — структура соединения;
- листинг 3.7 — структура плана HTTP-ответа;
- листинг 3.8 — структура результата отправки HTTP-ответа;
- листинг 3.9 — функция формирования плана HTTP-ответа;
- листинг 3.10 — функция подготовки файла для HTTP-ответа;
- листинг 3.11 — `makefile` для сборки и запуска сервера.

Листинг 3.1 — Алгоритм работы процесса-предка

```
1 #define DEFAULT_PORT 8080
2 #define DEFAULT_WORKERS 4
3 #define LISTEN_BACKLOG 128
4 #define DEFAULT_DOCROOT "./www"
5 #define DEFAULT_LOG_PATH "./server.log"
6 #define DEFAULT_ERROR_LOG_PATH "./server.error.log"
7 int main(int argc, char **argv) {
8     int port = DEFAULT_PORT;
```

```

9   int worker_count = DEFAULT_WORKERS;
10  const char *docroot = DEFAULT_DOCROOT;
11  const char *log_path = DEFAULT_LOG_PATH;
12  const char *error_log_path = DEFAULT_ERROR_LOG_PATH;
13  for (int i = 1; i < argc; ++i) {
14      if (strcmp(argv[i], "-p") == 0 && i + 1 < argc) {
15          if (parse_positive_number(argv[++i], &port) < 0 || port > 65535)
16              {
17                  fprintf(stderr, "Invalid port value\n");
18                  return 1;
19              }
20      } else if (strcmp(argv[i], "-w") == 0 && i + 1 < argc) {
21          if (parse_positive_number(argv[++i], &worker_count) < 0) {
22              fprintf(stderr, "Invalid workers value\n");
23              return 1;
24          }
25      } else if (strcmp(argv[i], "-d") == 0 && i + 1 < argc) {
26          docroot = argv[++i];
27      } else if (strcmp(argv[i], "-l") == 0 && i + 1 < argc) {
28          log_path = argv[++i];
29      } else if (strcmp(argv[i], "-e") == 0 && i + 1 < argc) {
30          error_log_path = argv[++i];
31      } else {
32          fprintf(stderr,
33              "Usage: %s [-p port] [-w workers] [-d docroot] [-l log_path] "
34              "[-e error_log_path]\n",
35              argv[0]);
36          return 1;
37      }
38  }
39  if (logger_set_error_path(error_log_path) < 0) {
40      fprintf(stderr, "Invalid error log path\n");
41      return 1;
42  }
43  if (logger_set_access_path(log_path) < 0) {
44      logger_log_error("master", "invalid access log path", EINVAL);
45      fprintf(stderr, "Invalid log path\n");
46      return 1;
47  }
48  if (worker_set_docroot(docroot) < 0) {
49      logger_log_error("master", "invalid docroot", EINVAL);

```

```

49     fprintf(stderr, "Invalid docroot\n");
50     return 1;
51 }
52 if (worker_set_log_path(log_path) < 0) {
53     logger_log_error("master", "failed to set access log path in worker
54         ", EINVAL);
55     fprintf(stderr, "Invalid log path\n");
56     return 1;
57 }
58 if (worker_set_error_log_path(error_log_path) < 0) {
59     logger_log_error("master", "failed to set error log path in worker"
60         , EINVAL);
61     fprintf(stderr, "Invalid error log path\n");
62     return 1;
63 }
64 int listen_fd = create_listen_socket(port);
65 if (listen_fd < 0) {
66     return 1;
67 }
68 pid_t *workers = calloc((size_t)worker_count, sizeof(pid_t));
69 if (workers == NULL) {
70     logger_log_error("master", "calloc failed", errno);
71     close(listen_fd);
72     return 1;
73 }
74 install_signal_handlers();
75 printf("[master %d] listening on port %d, workers=%d, docroot=%s, log
76     =%s, error_log=%s\n",
77     getpid(), port, worker_count, docroot, log_path, error_log_path);
78 fflush(stdout);
79 for (int i = 0; i < worker_count; ++i) {
80     pid_t pid = create_worker(listen_fd);
81     if (pid < 0) {
82         g_stop = 1;
83         worker_count = i;
84         break;
85     }
86     workers[i] = pid;
87 }
88 while (!g_stop) {
89     pause();

```

```

87     if (g_child_changed) {
88         g_child_changed = 0;
89         while (1) {
90             int status = 0;
91             pid_t dead = waitpid(-1, &status, WNOHANG);
92             if (dead <= 0) {
93                 break;
94             }
95             int slot = find_worker_slot(workers, worker_count, dead);
96             if (slot >= 0) {
97                 workers[slot] = -1;
98                 if (!g_stop) {
99                     pid_t replacement = create_worker(listen_fd);
100                     if (replacement > 0) {
101                         workers[slot] = replacement;
102                         printf("[master] respawn worker: %d -> %d\n", dead,
103                             replacement);
104                         fflush(stdout);
105                     } else {
106                         logger_log_error("master", "failed to respawn worker",
107                             errno);
108                         g_stop = 1;
109                     }
110                 }
111             }
112         }
113     }
114     printf("[master] stopping\n");
115     fflush(stdout);
116     for (int i = 0; i < worker_count; ++i) {
117         if (workers[i] > 0) {
118             kill(workers[i], SIGTERM);
119         }
120     }
121     while (waitpid(-1, NULL, 0) > 0) {}
122     close(listen_fd);
123     free(workers);
124     return 0;

```


Листинг 3.2 — Обработчики сигналов и функция их установки

```

1 static void on_stop_signal(int signo) {
2     (void)signo;
3     g_stop = 1;
4 }
5 static void on_sigchld(int signo) {
6     (void)signo;
7     g_chld_changed = 1;
8 }
9 static void install_signal_handlers(void) {
10     struct sigaction sa_stop;
11     memset(&sa_stop, 0, sizeof(sa_stop));
12     sa_stop.sa_handler = on_stop_signal;
13     sigemptyset(&sa_stop.sa_mask);
14     sigaction(SIGINT, &sa_stop, NULL);
15     sigaction(SIGTERM, &sa_stop, NULL);
16     struct sigaction sa_chld;
17     memset(&sa_chld, 0, sizeof(sa_chld));
18     sa_chld.sa_handler = on_sigchld;
19     sigemptyset(&sa_chld.sa_mask);
20     sa_chld.sa_flags = SA_RESTART | SA_NOCLDSTOP;
21     sigaction(SIGCHLD, &sa_chld, NULL);
22 }

```

Листинг 3.3 — Функция создания процессов-обработчиков

```

1 static pid_t create_worker(int listen_fd) {
2     pid_t pid = fork();
3     if (pid < 0) {
4         logger_log_error("master", "fork failed", errno);
5         return -1;
6     }
7     if (pid == 0) {
8         worker_loop(listen_fd);
9         close(listen_fd);
10        _exit(0);
11    }
12    return pid;
13 }

```

Листинг 3.4 — Функция создания слушающего сокета

```

1 static int create_listen_socket(int port) {
2     int fd = socket(AF_INET, SOCK_STREAM, 0);

```

```

3  if (fd < 0) {
4      logger_log_error("master", "socket failed", errno);
5      return -1;
6  }
7  int reuse = 1;
8  if (setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) <
9      0) {
10     logger_log_error("master", "setsockopt(SO_REUSEADDR) failed", errno
11     );
12     close(fd);
13     return -1;
14 }
15 struct sockaddr_in addr;
16 memset(&addr, 0, sizeof(addr));
17 addr.sin_family = AF_INET;
18 addr.sin_addr.s_addr = htonl(INADDR_ANY);
19 addr.sin_port = htons((uint16_t)port);
20 if (bind(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
21     logger_log_error("master", "bind failed", errno);
22     close(fd);
23     return -1;
24 }
25 if (listen(fd, LISTEN_BACKLOG) < 0) {
26     logger_log_error("master", "listen failed", errno);
27     close(fd);
28     return -1;
29 }

```

Листинг 3.5 — Функция работы worker'a

```

1 void worker_loop(int listen_fd) {
2     printf("[worker %d] started\n", getpid());
3     fflush(stdout);
4     connection_t *conns = calloc(FD_SETSIZE, sizeof(connection_t));
5     if (conns == NULL) {
6         logger_log_error("worker", "calloc connections failed", errno);
7         return;
8     }
9     for (int i = 0; i < FD_SETSIZE; ++i) {
10         connection_init(&conns[i]);
11     }

```

```

12  if (set_nonblocking(listen_fd) < 0) {
13      logger_log_error("worker", "failed to set listen socket nonblocking
        ", errno);
14  }
15  int max_fd = listen_fd;
16  while (!g_stop) {
17      fd_set readfds;
18      fd_set writefds;
19      FD_ZERO(&readfds);
20      FD_ZERO(&writefds);
21      FD_SET(listen_fd, &readfds);
22      for (int fd = 0; fd <= max_fd; ++fd) {
23          if (conns[fd].active) {
24              if (conns[fd].state == CONN_READING) {
25                  FD_SET(fd, &readfds);
26              } else {
27                  FD_SET(fd, &writefds);
28              }
29          }
30      }
31      int ready = select(max_fd + 1, &readfds, &writefds, NULL, NULL);
32      if (ready < 0) {
33          if (errno == EINTR) {
34              continue;
35          }
36          logger_log_error("worker", "select failed", errno);
37          break;
38      }
39      if (FD_ISSET(listen_fd, &readfds)) {
40          while (1) {
41              struct sockaddr_storage client_addr;
42              socklen_t client_len = sizeof(client_addr);
43              int client_fd = accept(listen_fd, (struct sockaddr *)&
                client_addr, &client_len);
44              if (client_fd < 0) {
45                  if (errno == EAGAIN || errno == EWOULDBLOCK) {
46                      break;
47                  }
48                  if (errno != EINTR) {
49                      logger_log_error("worker", "accept failed", errno);
50                  }

```

```

51         break;
52     }
53
54     if (client_fd >= FD_SETSIZE) {
55         close(client_fd);
56         continue;
57     }
58     if (set_nonblocking(client_fd) < 0) {
59         logger_log_error("worker", "failed to set client socket
60             nonblocking", errno);
61         close(client_fd);
62         continue;
63     }
64     connection_init(&conns[client_fd]);
65     conns[client_fd].active = true;
66     conns[client_fd].state = CONN_READING;
67     conns[client_fd].addr = client_addr;
68     conns[client_fd].addr_len = client_len;
69     if (client_fd > max_fd) {
70         max_fd = client_fd;
71     }
72 }
73 for (int fd = 0; fd <= max_fd; ++fd) {
74     if (!conns[fd].active) {
75         continue;
76     }
77     if (conns[fd].state == CONN_READING && FD_ISSET(fd, &readfds)) {
78         size_t left = sizeof(conns[fd].request_buf) - 1 - conns[fd].
79             request_len;
80         if (left == 0) {
81             if (prepare_connection_response(&conns[fd]) < 0) {
82                 logger_log_error("worker", "prepare response failed for
83                     full request buffer",
84                     errno);
85                 close_connection(fd, &conns[fd]);
86             }
87             continue;
88         }
89         ssize_t n = recv(fd, conns[fd].request_buf + conns[fd].
90             request_len, left, 0);

```

```

88     if (n < 0) {
89         if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR
90             ) {
91             continue;
92         }
93         logger_log_error("worker", "recv failed", errno);
94         close_connection(fd, &conns[fd]);
95         continue;
96     }
97     if (n == 0) {
98         close_connection(fd, &conns[fd]);
99         continue;
100     }
101     conns[fd].request_len += (size_t)n;
102     conns[fd].request_buf[conns[fd].request_len] = '\0';
103     if (request_line_complete(&conns[fd])) {
104         if (prepare_connection_response(&conns[fd]) < 0) {
105             logger_log_error("worker", "http_prepare_response failed",
106                 errno);
107             close_connection(fd, &conns[fd]);
108         }
109     }
110     if (conns[fd].active && conns[fd].state == CONN_WRITING &&
111         FD_ISSET(fd, &writefds)) {
112         if (conns[fd].header_sent < conns[fd].plan.header_len) {
113             const char *h = conns[fd].plan.header + conns[fd].header_sent
114                 ;
115             size_t len = conns[fd].plan.header_len - conns[fd].
116                 header_sent;
117             ssize_t n = send(fd, h, len, MSG_NOSIGNAL);
118             if (n < 0) {
119                 if (errno != EAGAIN && errno != EWOULDBLOCK && errno !=
120                     EINTR) {
121                     logger_log_error("worker", "send header failed", errno);
122                     close_connection(fd, &conns[fd]);
123                 }
124                 continue;
125             }
126             conns[fd].header_sent += (size_t)n;
127         }
128     }

```

```

123     if (conns[fd].header_sent == conns[fd].plan.header_len && !
124         conns[fd].body_done) {
125         if (conns[fd].write_buf_sent == conns[fd].write_buf_len) {
126             ssize_t nread = read(conns[fd].plan.file_fd, conns[fd].
127                 write_buf,
128                 sizeof(conns[fd].write_buf));
129             if (nread < 0) {
130                 if (errno == EINTR) {
131                     continue;
132                 }
133                 logger_log_error("worker", "read file chunk failed",
134                     errno);
135                 close_connection(fd, &conns[fd]);
136                 continue;
137             }
138             if (nread == 0) {
139                 conns[fd].body_done = true;
140             } else {
141                 conns[fd].write_buf_len = (size_t)nread;
142                 conns[fd].write_buf_sent = 0;
143             }
144         }
145         if (!conns[fd].body_done && conns[fd].write_buf_sent < conns[
146             fd].write_buf_len) {
147             const char *p = conns[fd].write_buf + conns[fd].
148                 write_buf_sent;
149             size_t len = conns[fd].write_buf_len - conns[fd].
150                 write_buf_sent;
151             ssize_t n = send(fd, p, len, MSG_NOSIGNAL);
152             if (n < 0) {
153                 if (errno != EAGAIN && errno != EWOULDBLOCK && errno !=
154                     EINTR) {
155                     logger_log_error("worker", "send body chunk failed",
156                         errno);
157                     close_connection(fd, &conns[fd]);
158                 }
159                 continue;
160             }
161             conns[fd].write_buf_sent += (size_t)n;
162             conns[fd].body_bytes_sent += n;
163         }
164     }

```

```

156     }
157     if (conns[fd].active && conns[fd].header_sent == conns[fd].plan
        .header_len &&
158     conns[fd].body_done) {
159         finalize_and_close_connection(fd, &conns[fd]);
160     }
161 }
162 }
163 while (max_fd > listen_fd && !conns[max_fd].active) {
164     --max_fd;
165 }
166 }
167 for (int fd = 0; fd <= max_fd; ++fd) {
168     if (conns[fd].active) {
169         close_connection(fd, &conns[fd]);
170     }
171 }
172 free(conns);
173 printf("[worker %d] stopping\n", getpid());
174 fflush(stdout);
175 }

```

Листинг 3.6 — Структура соединения

```

1 typedef struct connection {
2     bool active;
3     connection_state_t state;
4     struct sockaddr_storage addr;
5     socklen_t addr_len;
6     char request_buf[READ_BUFFER_SIZE];
7     size_t request_len;
8     http_response_plan_t plan;
9     size_t header_sent;
10    char write_buf[WRITE_BUFFER_SIZE];
11    size_t write_buf_len;
12    size_t write_buf_sent;
13    bool body_done;
14    bool timer_started;
15    struct timespec started_at;
16    long long body_bytes_sent;
17 } connection_t;

```

Листинг 3.7 — Структура плана HTTP-ответа

```

1 typedef struct http_response_plan {
2     char method[16];
3     char uri[2048];
4     int status_code;
5     char header[1024];
6     size_t header_len;
7     bool send_body;
8     int file_fd;
9     off_t file_size;
10 } http_response_plan_t;

```

Листинг 3.8 — Структура результата отправки HTTP-ответа

```

1 typedef struct http_result {
2     char method[16];
3     char uri[2048];
4     int status_code;
5     long long bytes_sent;
6 } http_result_t;

```

Листинг 3.9 — Функция подготовки плана ответа

```

1 int http_prepare_response(const char *raw, const char *docroot,
2     http_response_plan_t *plan) {
3     char method[32];
4     char uri[2048];
5     char version[32];
6     char path[PATH_MAX];
7     bool is_head = false;
8     if (plan == NULL) {
9         return -1;
10    }
11    plan_init(plan);
12    if (parse_request_line(raw, method, sizeof(method), uri, sizeof(uri),
13        version,
14        sizeof(version)) < 0) {
15        return build_error_header(plan, 400);
16    }
17    copy_trunc(plan->method, sizeof(plan->method), method);
18    copy_trunc(plan->uri, sizeof(plan->uri), uri);
19    if (strcmp(method, "GET") == 0) {
20        is_head = false;
21    } else if (strcmp(method, "HEAD") == 0) {

```



```

20     is_head = true;
21 } else {
22     return build_error_header(plan, 405);
23 }
24
25 if (resolve_path(docroot, uri, path, sizeof(path)) < 0) {
26     return build_error_header(plan, 403);
27 }
28
29 return prepare_file_response(path, is_head, plan);
30 }

```

Листинг 3.10 — Функция подготовки файла

```

1 static int prepare_file_response(const char *path, bool is_head,
  http_response_plan_t *plan) {
2     struct stat st;
3     if (stat(path, &st) < 0) {
4         if (errno == EACCES) {
5             return build_error_header(plan, 403);
6         }
7         return build_error_header(plan, 404);
8     }
9     if (!S_ISREG(st.st_mode)) {
10         return build_error_header(plan, 403);
11     }
12     if (access(path, R_OK) < 0) {
13         return build_error_header(plan, 403);
14     }
15     if (st.st_size < 0 || st.st_size > MAX_FILE_SIZE) {
16         return build_error_header(plan, 403);
17     }
18     const char *content_type = guess_content_type(path);
19     int n = snprintf(plan->header, sizeof(plan->header),
20 "HTTP/1.1 200 OK\r\nConnection: close\r\nContent-Type: %s\r\n"
21 "Content-Length: %lld\r\n\r\n",
22 content_type, (long long)st.st_size);
23     if (n <= 0 || (size_t)n >= sizeof(plan->header)) {
24         return -1;
25     }
26     plan->header_len = (size_t)n;
27     plan->status_code = 200;
28     plan->file_size = st.st_size;

```

```

29     if (is_head) {
30         plan->send_body = false;
31         plan->file_fd = -1;
32         return 0;
33     }
34     int fd = open(path, O_RDONLY);
35     if (fd < 0) {
36         return build_error_header(plan, 403);
37     }
38     plan->file_fd = fd;
39     plan->send_body = true;
40     return 0;
41 }

```

Листинг 3.11 — Makefile для сборки и запуска сервера

```

1 CC := gcc
2 CFLAGS := -std=c11 -Wall -Wextra -Wpedantic -O2
3 LDFLAGS :=
4 TARGET := server
5 SRC := src/main.c src/worker.c src/http.c src/log.c
6 OBJ := $(SRC:.c=.o)
7 .PHONY: all clean run
8 all: $(TARGET)
9 $(TARGET): $(OBJ)
10 $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)
11 %.o: %.c
12 $(CC) $(CFLAGS) -c -o $@ $<
13 run: $(TARGET)
14 ./$(TARGET)
15 clean:
16 rm -f $(TARGET) $(OBJ)

```

3.3 Демонстрация работы алгоритма

По умолчанию сервер возвращает HTML-страницу с CSS-стилями. В листинге 3.12 показан результат работы обработки запроса с методом GET, в листинге 3.13 — запроса с методом HEAD, в листинге 3.14 — запроса с не обрабатываемым методом POST.

Листинг 3.12 — Запрос GET

```

1 $ curl -i http://127.0.0.1:8080/
2 HTTP/1.1 200 OK

```

```

3 Connection: close
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 424
6
7 <!doctype html>
8 <html lang="en">
9 <head>
10   <meta charset="utf-8">
11   <meta name="viewport" content="width=device-width, initial-scale=1">
12   <title>Static Server</title>
13   <link rel="stylesheet" href="/style.css">
14 </head>
15 <body>
16   <main class="card">
17     <h1>Static HTTP server</h1>
18     <p>The server is running and serves files from docroot.</p>
19     <p>Current MVP supports GET and HEAD methods.</p>
20   </main>
21 </body>
22 </html>

```

Листинг 3.13 — Запрос HEAD

```

1 $ curl -I http://127.0.0.1:8080/
2 HTTP/1.1 200 OK
3 Connection: close
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 424

```

Листинг 3.14 — Запрос HEAD

```

1 $ curl -i -X POST http://127.0.0.1:8080/
2 HTTP/1.1 405 Method Not Allowed
3 Connection: close
4 Allow: GET, HEAD
5 Content-Length: 0

```

3.4 Тестирование

В таблице 3.1 приведены тестовые сценарии и результаты тестирования разработанного HTTP-сервера.

Таблица 3.1 — Тестирование разработанного HTTP-сервера

Тест	Тестовый сценарий	Ожидаемый результат	Полученный результат
Запрос GET для получения файла по умолчанию	\$ curl -i http://127.0.0.1:8080/	200 OK, файл в теле ответа	200 OK, файл в теле ответа
Запрос GET для получения файла	\$ curl -i http://127.0.0.1:8080/manual_test.txt	200 OK, файл в теле ответа	200 OK, файл в теле ответа
Запрос HEAD	\$ curl -I http://127.0.0.1:8080/manual_test.txt	200 OK	200 OK
Запрос GET файла размером до 128 МБ	\$ curl -i http://127.0.0.1:8080/large_120mb.txt	200 OK, файл в теле ответа	200 OK, файл в теле ответа
Запрос POST не поддерживаемого метода	\$ curl -i -X POST http://127.0.0.1:8080/	405 Method Not Allowed	405 Method Not Allowed
Запрос GET файла размером свыше 128 МБ	\$ curl -i http://127.0.0.1:8080/large_129mb.txt	403 Forbidden, пустое тело ответа	403 Forbidden, пустое тело ответа
Запрос GET несуществующего файла	\$ curl -i http://127.0.0.1:8080/not_found.txt	404 Not Found, пустое тело ответа	404 Not Found, пустое тело ответа
Запрос GET с path traversal	\$ curl -i "http://127.0.0.1:8080../etc/passwd"	403 Forbidden, пустое тело ответа	403 Forbidden, пустое тело ответа
Запрос GET директории	\$ curl -i http://127.0.0.1:8080/dir	403 Forbidden, пустое тело ответа	403 Forbidden, пустое тело ответа
Запрос GET с невалидной стартовой строкой	\$ printf 'BROKEN_REQUEST\r\n\r\n' nc 127.0.0.1 8080	400 Bad Request, пустое тело ответа	400 Bad Request, пустое тело ответа

Вывод

В данном разделе определены средства реализации, приведены реализации основных алгоритмов и функций, продемонстрирована работа HTTP-сервера и проведено его тестирование.

4 Исследовательская часть

В данном разделе поставлена задача исследования зависимостей среднего времени выполнения запроса, числа запросов в секунду, средней скорости передачи данных и средней скорости передачи данных на одно соединение от количества активных соединений.

4.1 Характеристики устройства

Исследования проводились на машине со следующими характеристиками:

- процессор Intel(R) Core(TM) i5-10210U, тактовая частота 1.60 ГГц, 8 логических ядер;
- оперативная память: 16 ГБ;
- операционная система: Ubuntu 22.04.4 LTS.

4.2 Методика исследования

Для проведения исследования использовалась утилита wrk [10], позволяющая получить необходимые метрики при заданном числе активных соединений. Параметры тестового прогона:

- адрес сервера: localhost:8080;
- размер запрашиваемого ресурса: 5 МБ;
- количество создающих нагрузку потоков: 4;
- число активных соединений: от 50 до 1050 или первой ошибки запроса с шагом 100;
- число процессов-обработчиков: 4, 8, 16;
- длительность нагрузки при заданном числе обработчиков и активных соединений: 8 с;
- количество замеров при заданном числе обработчиков и активных соединений: 10.

4.3 Результаты

Основным ограничением максимального числа открытых соединений является ограничение на номера файловых дескрипторов FD_SETSIZE (1024). Таким образом при 1050 активных соединениях происходит 31 ошибка установки соединения, из-за недостаточного количества номеров свободных файловых дескрипторов.

Полученные зависимости представлены на графиках:

- рисунок 4.1 — зависимость среднего времени числа выполнения запроса от числа активных соединений;
- рисунок 4.2 — зависимость RPS от числа активных соединений;

— рисунок 4.3 — зависимость средней скорости передачи данных от числа активных соединений;

— рисунок 4.4 — зависимость средней скорости передачи данных на одно соединение от числа активных соединений.

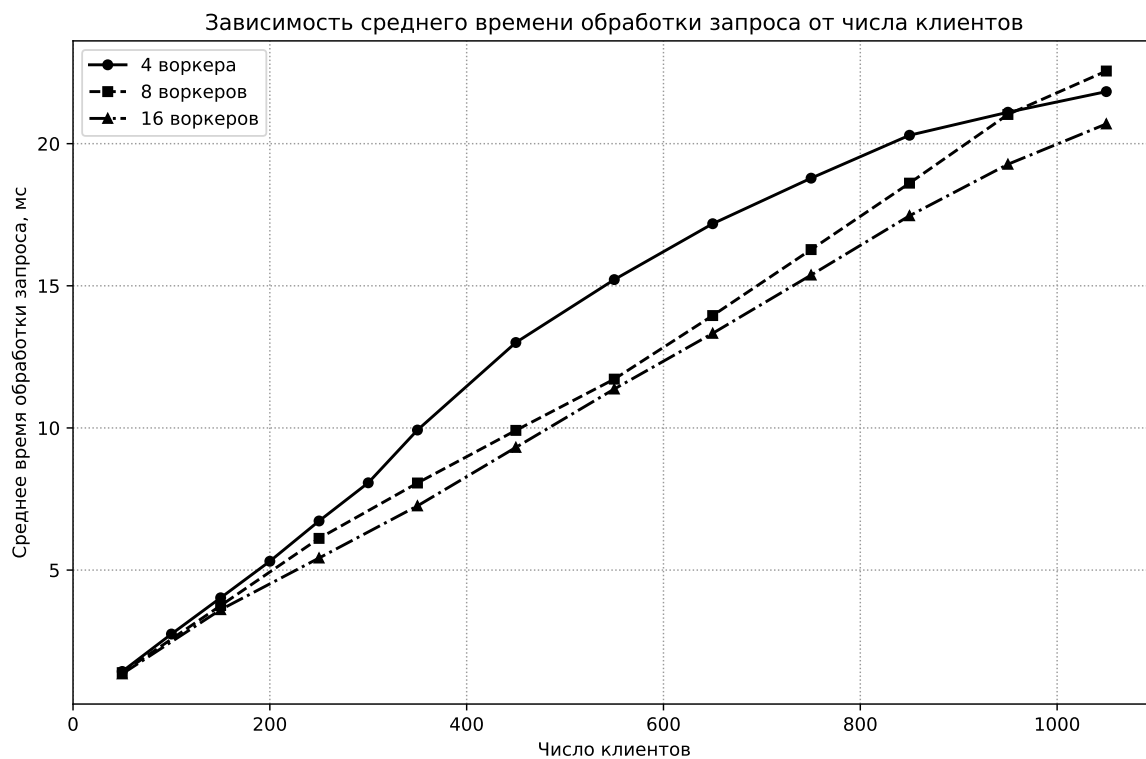


Рисунок 4.1 — Зависимость среднего времени выполнения запроса от числа активных соединений

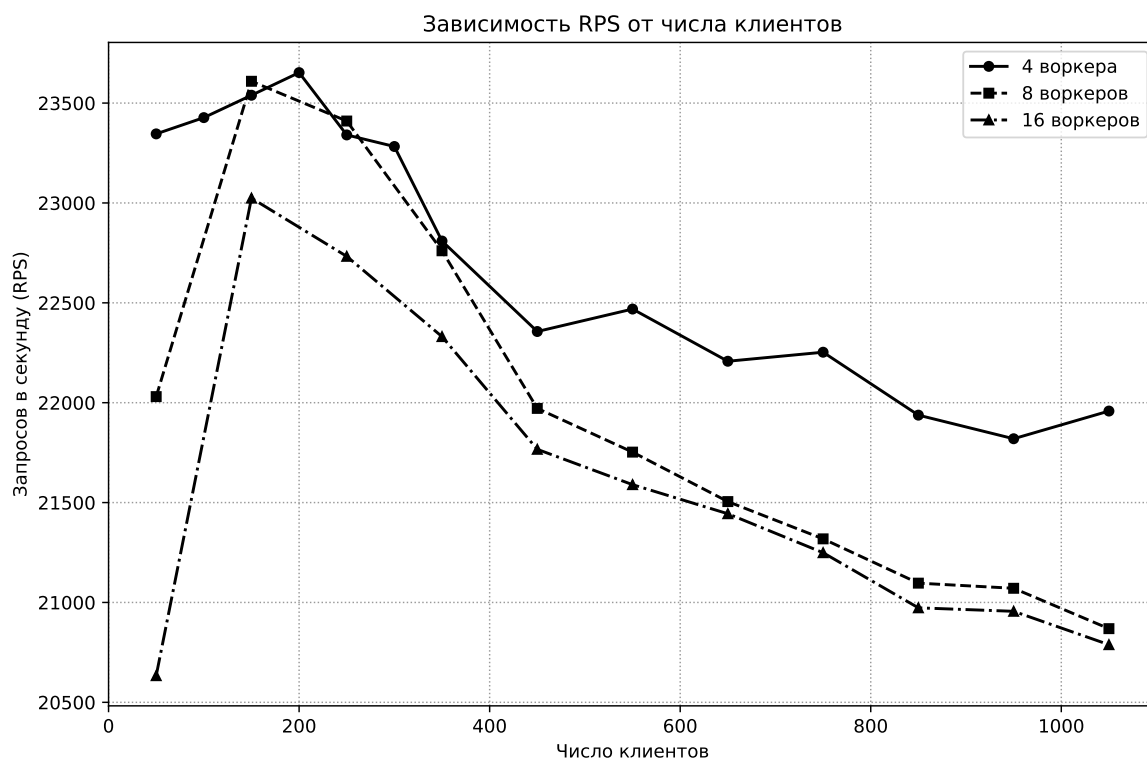


Рисунок 4.2 — Зависимость RPS от числа активных соединений

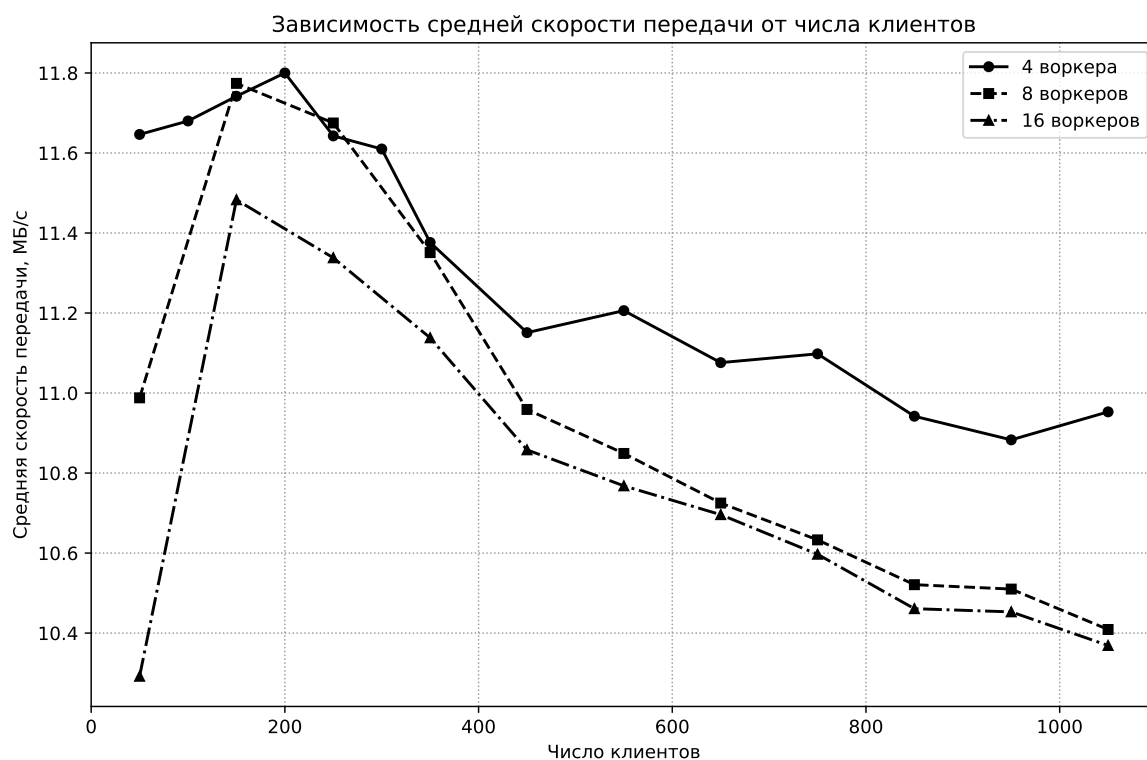


Рисунок 4.3 — Зависимость средней скорости передачи данных от числа активных соединений

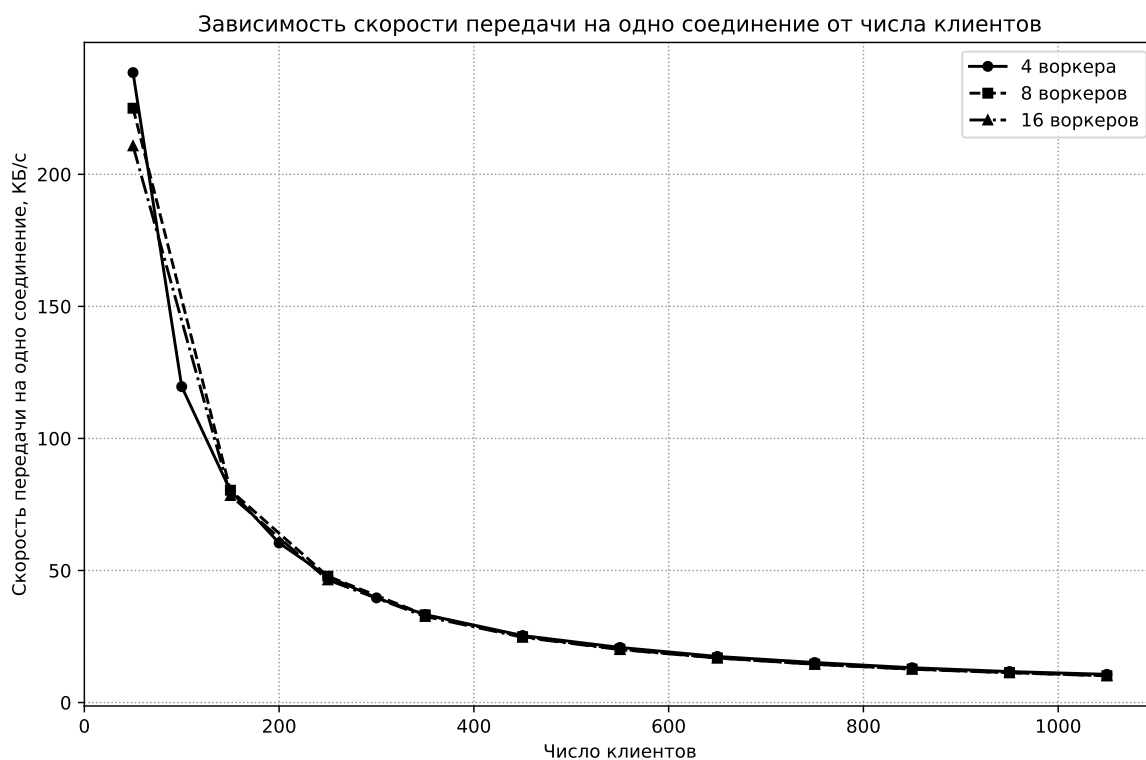


Рисунок 4.4 — Зависимость средней скорости передачи данных на одно соединение от числа активных соединений

4.4 Вывод

По результатам замеров исследования можно сделать следующие выводы:

- среднее время выполнения запроса увеличивается с ростом числа активных соединений, от менее 5 мс при 50 соединениях до приблизительно 23 мс при 1000 соединениях;
- минимальная скорость роста времени выполнения запроса наблюдается при 16 процессах-обработчиках;
- расхождение во времени выполнения процесса при 1000 активных соединениях между максимальным значением при 4 обработчиках и минимальным при 16 составляет не более 3 мс;
- количество обрабатываемых запросов в секунду уменьшается с ростом числа активных соединений от порядка 23000 RPS при 150 соединениях до порядка 20800 RPS при 1000 соединениях;
- минимальное значение RPS при 1000 активных соединений наблюдается при 16 процессах-обработчиках, максимальное — при 4;
- средняя скорость передачи данных также уменьшается с ростом числа активных соединений от порядка 11.5 МБ/с при 150 соединениях до 10.4 МБ/с при 1000 соединениях;
- максимальное значение скорости передачи данных при 1000 активных соединениях наблюдается при 4 процессах-обработчиках, минимальное при 16;
- средняя скорость передачи данных на одно соединение убывает с ростом числа ак-

тивных соединений от порядка 210 КБ/с при 50 соединениях до порядка 10 КБ/с при 1000 соединениях;

— различия в скорости передачи данных на одно соединение при количестве активных соединений свыше 250 не превышает 2 КБ/с.

ЗАКЛЮЧЕНИЕ

Поставленная цель была достигнута: разработан HTTP-сервер для отдачи статического содержимого с использованием мультиплексирования соединений на основе системного вызова `select()` и предварительно созданных процессов-обработчиков, обеспечивающих параллельную обработку соединений.

В ходе выполнения работы были выполнены следующие задачи:

- выполнен анализ предметной области: основные принципы устройства и работы протокола HTTP, механизм мультиплексирования с использованием системного вызова `select()` и подход к организации параллельной обработки соединений;
- спроектированы сущности базы данных и определены ограничения целостности данных, спроектирована ролевую модель на уровне базы данных;
- выбраны средства реализации и реализован HTTP-сервер;
- проведено исследование и определено: максимальное количество обслуживаемых сетевых соединений, зависимость скорости передачи данных по каждому сетевому соединению, совокупной средней скорости передачи и среднего времени обработки запроса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Canonical Ltd. Ubuntu Desktop Help.
<https://help.ubuntu.com/22.04/ubuntu-help/index.html>, 2022. [Электронный ресурс] Дата обращения: 03.03.26.
2. R. Fielding, M. Nottingham, and J. Reschke. Http semantics. RFC 9110, Section 9.
<https://www.rfc-editor.org/rfc/rfc9110#name-methods>, 2022. [Электронный ресурс] Дата обращения: 03.03.26.
3. R. Fielding, M. Nottingham, and J. Reschke. Http/1.1. RFC 9112.
<https://datatracker.ietf.org/doc/html/rfc9112>, 2022.
4. Free Software Foundation. GNU C Library Manual.
<https://glibcdocs.readthedocs.io/en/latest/>, 2016. [Электронный ресурс]. Дата обращения: 03.03.26.
5. ISO/IEC JTC 1/SC 22. Information technology — programming languages — c. International Standard ISO/IEC 9899:2011, International Organization for Standardization, 2011. Status: Withdrawn. Дата обращения: 03.03.26.
6. Linux man-pages project. accept(2) — Linux manual page.
<https://man7.org/linux/man-pages/man2/accept.2.html>, 2025. [Электронный ресурс] Дата обращения: 03.03.26.
7. Linux man-pages project. fork(2) — Linux manual page.
<https://man7.org/linux/man-pages/man2/fork.2.html>, 2025. [Электронный ресурс] Дата обращения: 03.03.26.
8. Linux man-pages project. select(2) — Linux manual page.
<https://man7.org/linux/man-pages/man2/select.2.html>, 2025. [Электронный ресурс] Дата обращения: 03.03.26.
9. MDN Web Docs Contributors. Обзор протокола http.
<https://developer.mozilla.org/ru/docs/Web/HTTP/Guides/Overview>, 2025.
[Электронный ресурс] Дата обращения: 03.03.26.
10. Ubuntu Manual Pages. wrk(1) — modern HTTP benchmarking tool.
<https://manpages.ubuntu.com/manpages/noble/man1/wrk.1.html>, 2024.
[Электронный ресурс] Дата обращения: 03.03.26.